



Integration Guide v5.5

Contents

1	Document Approval	4
2	Version History	5
3	Disclaimer	8
4	References	9
5	Acronyms	10
6	Introduction	11
7	SoftPOS solution overview	12
7.1	Technical content	12
7.2	Attestation mechanism	12
7.2.1	Enrollment	12
7.2.2	Attestation	13
8	Prerequisites	14
8.1	Baseline Platforms	14
8.2	Permissions	14
8.3	ALCINEO SoftPOS libraries dependencies	14
8.4	Application mandatory dependencies	15
8.5	Assets	15
9	SoftPOS initialisation setup	16
9.1	Basic example	16
9.2	Libraries initialization	17
9.3	Update Initial Session Key	18
9.4	Enrollment	18
9.5	TR34 Remote Key Injection	18
9.5.1	Purpose	18
9.5.2	TR34 Description	20
9.5.2.1	Flow	20
9.5.2.2	TR34 recommended CA scheme	21
9.5.2.3	TR-34 replay prevention	21
9.5.3	TR34 API	22
9.5.3.1	Call sequence	22
9.5.3.2	API for option 1	23
9.5.3.3	API for option 2	23
9.5.4	Key Bundle Format	24
9.5.5	Example of TR31 Key Generation After TR34 TR_IMEAK Injection	24
9.5.6	Example of TR31 Key Block with TR34 IMEAK Injection	25

10 Performing a transaction	27
10.1 Start transaction	27
10.1.1 TransactionParameters	28
10.2 Transaction (and Pin) Events	28
11 Sensitive Data Management	31
11.1 Smart Card Sensitive Data	31
11.2 Sensitive Data Encryption	31
11.3 Outcome Example	32
11.3.1 Encrypted data	32
11.3.2 Signature	32
11.3.3 DUKPT Keys Diversification	32
11.3.4 Data decryption	33
11.3.5 Signature verification	34
12 PIN Management	35
12.1 Introduction	35
12.2 PIN encryption	35
12.2.1 ISO 9564-1 Format 4	35
12.2.2 PIN decryption example	36
13 Audit Logs	39
13.1 Description	39
13.2 Signature	39
13.3 Send of Audit Logs	39
13.3.1 The SDK	40
13.3.2 The application	40
13.3.3 The A&M server	40
13.4 Audit logs Analysis	40
13.4.1 Routine messages	41
13.4.2 Selftests problems	41
13.4.3 Key management problems	42
13.4.4 Key injection problems	42
13.4.5 Attestation problems	42
13.4.6 Cipherring problems	43
13.4.7 Tamper problems	43
13.4.8 Signature problem	43
14 Error Management	45
15 Libraries protection	50
15.1 DUKPT keys size	50
15.2 Active Security Checks	50
15.3 Code Obfuscation	50
15.4 White-boxes	51

15.5	SSL Keys	51
16	Security Guideline	52
16.1	Platform baseline restriction	52
16.2	Security options	52
16.3	Application signing	52
16.4	Sensitive data deletion	53
16.5	NFC Usage	53
16.6	Files protection	53
16.7	Application Backup Forbidden	53
16.8	Developer mode protection	54
16.9	Screenshot Protection	54
16.10	Camera Protection	54
16.11	Random Generator Initialization	55
16.12	Terminal settings	55
16.13	Attestation system	55
16.13.1	Attestation ID	55
16.13.2	Periodic Attestation	56
16.13.3	Device Check	56
16.13.4	Incident API	56
16.13.5	Communication with servers	60
16.13.6	Applicative protection	60
16.13.7	Android Version Whitelist	60
16.13.8	Production Version Blacklist	61
16.14	SideLoading protection	61
16.15	Repackaging protection	61
17	Key details	62
18	Secure Software Configuration	66
19	Host to Host	70
19.1	Attestation ID API	70

1 Document Approval

Role	Name	Date	Signature
CEO	Arnaud Corria	January, 2025	
CTO	Olivier Capron	January, 2025	
Security Manager	Gilles <u>Letourneux</u>	January, 2025	
Development Engineer	Franck Renault	January, 2025	
Development Engineer	Thibaut Barthe	January, 2025	

2 Version History

This documentation is delivered with SDK-7.4.6 package delivery.

Table 1: Revision History

Version	Date	Revision Description
3.0	August, 2023	New protocol keys
3.1	Sept, 2023	Additional advice for MPOC
3.2	Dec, 2023	Added chapter <i>Application Backup Forbidden</i> ; build update done every 4 weeks; updated information on audit logs and incidents; added FK_NONCE; deleted FK_RAK; random seed increased to 16 bytes.
3.3	Dec, 2023	Server error codes list updated; Certificate Update interface added in init API description.
3.4	Jan, 2024	Server error codes list split into 2 tables and new code added; clarification on audit log signature problems. Added <i>Device Check</i> paragraph. Added Blacklist management. Changed initialization order. Added A&M protocol doc link in <i>Communication with A&M Server</i> section. Magstripe reference removed. Non-isolated SDK. Developer mode managed by SDK. FK_KEK replaces FK_KEK_SES.
3.5	Feb, 2024	Random seed update.
3.6	April, 2024	Changed AES key size to 128 bits; updated examples.
3.7	June, 2024	FLAG_SECURE managed by SDK.
3.8	July, 2024	Added incident codes; seed length increased to 24 bytes; added DRBG info.
3.9	Nov, 2024	HSM seed source made FIPS or PCI compliant; added protection against camera.

Continued on next page

Table 1 – continued from previous page

Version	Date	Revision Description
4.0	Dec, 2024	Formatted Incident table; more detailed key distribution scheme; updated References.
4.1	Feb, 2025	Added new server error codes; updated payment data encryption with unpredictable IV.
4.2	April, 2025	Updated key injection mechanism (TR34); changed mechanism of setSeed; updated information about TLS mutual authentication.
4.3	May, 2025	Merging Integration guides in this one and reorganize it.
4.4	May, 2025	Forbidden the HTTP protocol for the final application in its manifest file.
4.5	June, 2025	Update PIN management and recommendation of TLS minimal version to 1.3.
4.6	June, 2025	Restrict to devices supporting secure hardware key storage
4.7	June, 2025	Change AttestationAgent init
4.8	June, 2025	Add application signature requirement and precise the restriction of android version
4.9	June, 2025	Add security configuration section to justify the absence of security options in the SDK.
5.0	June, 2025	Report the responsibility of the application developer on side-loading and repackaging protection.
5.1	July, 2025	change whitelist responsibility
5.2	July, 2025	Add a section for Secure Configuration
5.3	August, 2025	Review explanation of TR34 injection option 2, add ephemeral keys in key table.

Continued on next page

Table 1 – continued from previous page

Version	Date	Revision Description
5.4	August, 2025	Restrict android version to android 13
5.5	September, 2025	Delete the case PAN not present in PIN deciphering example, use tag 57.

3 Disclaimer

National legislation and international treaties regarding copyright protect this material. The name ALCINEO, its logo, software products, service names and their respective logos are registered trademarks or service marks of ALCINEO, a private company registered under number 497 965 152 of the Trade & Company Registry of Marseille, whose head offices are located at ZI Athélia IV, 233 Avenue du Mistral, 13600 La Ciotat, France.

When they are not registered trademarks, the names of software products published by ALCINEO are protected by national and international trade name legislation and by national and international software legislation. The other cited brand names and registered trademarks are the property of their respective owners.

No part of this material may be reproduced, in any form or by any means, without prior written consent of ALCINEO, unless otherwise stated in a written agreement executed by ALCINEO. In no event shall ALCINEO be liable for any typographical errors, screenshot errors, or any consequences arising from incorrect use of this material.

This material is intended exclusively for informational, educational and/or training purposes for its recipients. It can in no way be considered as a contract, agreement (including *sui generis*), or an advertising and/or promotional text.

This material may be stored in a digital format and therefore may be subject to alteration. ALCINEO shall not accept responsibility for any alteration or falsification of this material.

© 2025 ALCINEO – All rights reserved.

4 References

- Retail Financial Services **Symmetric Key Management Part 3: Using Symmetric Techniques**, ANSI X9.24 Part 3 [1]
- ANSI X9 TR31-2018 **Interoperable Secure Key Exchange Key Block Specification** [2]
- Alcineo document, **SoftPOS Integration Guide** [3]
- ANSI X9 TR34-2012 **Interoperable Method for Distribution of Symmetric Keys using Asymmetric** [4]

5 Acronyms

Abbreviation	Description
APK_KEK	Application Key Encrypting Key
COTS	Commercial Off-The-Shelf
DUKPT	Derived Unique Key Per Transaction
FK_AK	Factory Key Attestation (for A&M protocol type A commands)
FK_IPEK	Factory Key IPEK (for TR31 injection of IPEK)
FK_NONCE	Factory Key for NONCE encryption (A&M protocol)
IMEAK	Initial Message Encryption/Authentication Key (initial DUKPT)
KSN	Key Serial Number
MAK	Message Authentication Key
MEAK	Message Encryption/Authentication Key (DUKPT session parent)
MEK	Message Encryption Key
MPOC	PCI Mobile Payments on COTS
POS	Point Of Sale
PIN	Personal Identification Number
PAN	Primary Account Number
PCI	Payment Card Industry
PCI DSS	PCI Data Security Standard
RAND_KEY	Random Key
SDK	Software Development Kit

6 Introduction

The ALCINEO SoftPOS solution is a software-based payment solution that allows merchants to accept card payments using their mobile devices.

It is designed to be compliant with the PCI Mobile Payments on COTS (MPOC) standard, which ensures that the solution meets the necessary security requirements for processing card payments.

Our solution enforces critical security measures by default, minimizing the need for end-user configuration. At this time, there are no significant user-configurable security parameters beyond the defaults set in the codebase. Therefore, our documentation does not include extensive configurable security options or parameters.

Should future releases introduce any new security-related configurations (e.g., adjustable encryption settings, custom policy files, logging levels, or access controls), we will:

- Clearly document each option or parameter in the Integration Guide or a dedicated Security Configuration Guide.
- Explain its purpose, recommended values, and any potential security impacts.
- Provide instructions and best practices for secure deployment and maintenance.

This approach ensures that, even without extensive configurable options, we maintain a robust security posture by design. If custom configurations are ever required for specific environments, our team will provide tailored guidance and ensure the security implications are fully documented.

This document provides guidelines for integrating the ALCINEO SoftPOS libraries into your application and describes the security features of the ALCINEO SoftPOS solution.

This document is not merely educational material. It is important to emphasize that, **in order to comply with MPOC requirements, the security recommendations outlined in this document must be strictly adhered to.**

7 SoftPOS solution overview

7.1 Technical content

The libraries mandatory to integrate in your application are:

- SDK-*-Payment-*.aar: This library includes the payment kernels provided by ALCINEO, for instance MasterCard MCL 3.1.3, VISA Paywave 2.2, American Express expresspay 4.0.3, Discover DP 2.0.0, JCB 1.5.0, Union Pay CUP 2.0.21.
- Softpos-*.aar: This library is the layer used by your android application to manage transaction
- Softpos-*.javadoc.jar: This is the javadoc library for SoftPOS API

In the delivery package, all libraries are available in 2 modes, “protected” or “unprotected”

- The “protected” label in library name means the library contains countermeasure (root and debug detection)
- The “unprotected” label in library name means the library does not contain countermeasures

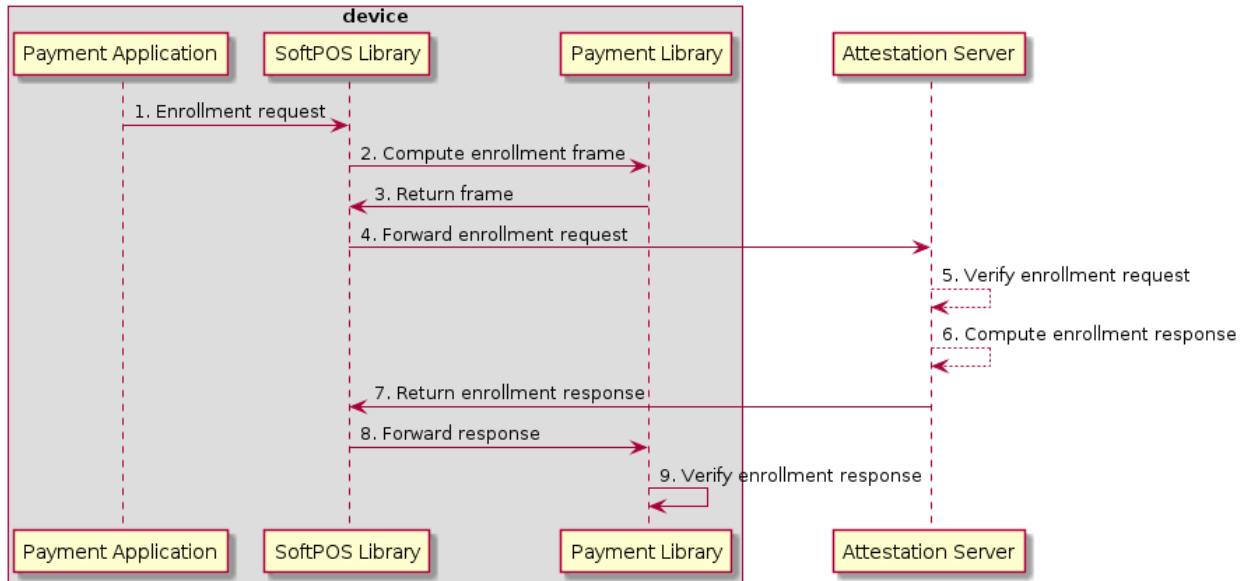
For you final application please use “protected” libraries (During production phase only these libraries will be delivered). But for your development phase, please use “unprotected” libraries. With these one, you can debug your android application.

7.2 Attestation mechanism

7.2.1 Enrollment

In the ALCINEO SoftPOS solution, an **enrollment** must be performed with the Attestation Server during the first initialization of the device.

The purpose of enrollment is to securely identify a device to the A&M Server and store the device information in a database.



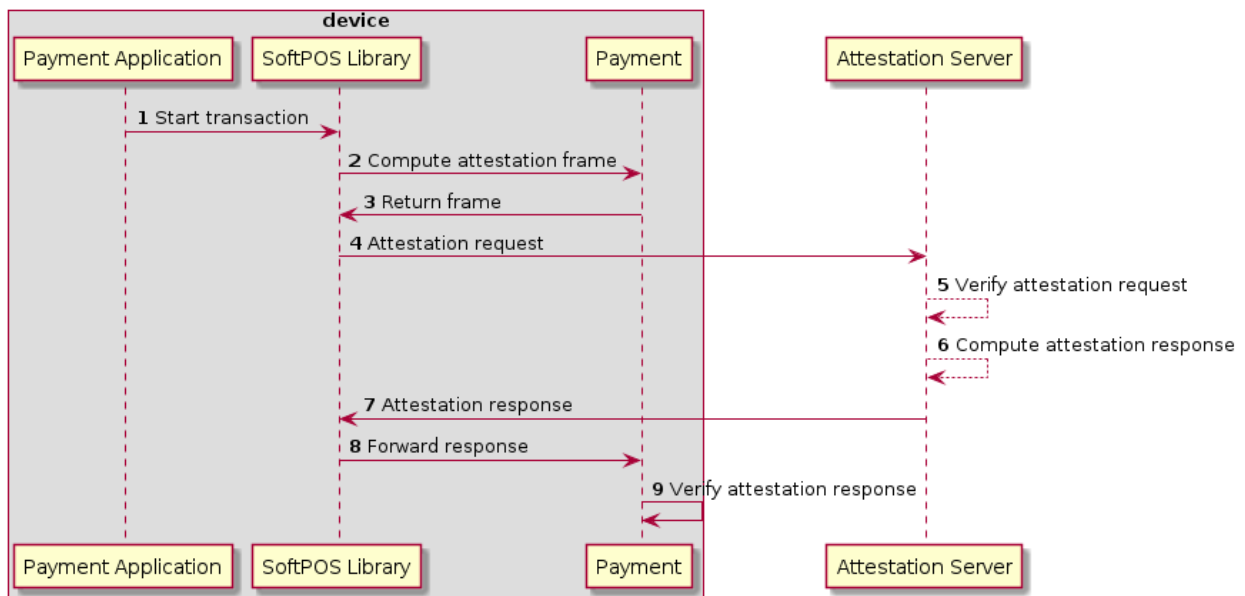
7.2.2 Attestation

During a transaction, an attestation request is executed once at the beginning of the transaction.

Even if an API exist to force an attestation request, during a transaction this action is handled by the SoftPOS library.

The attestation request certifies that the device has the right to process a transaction. If the enrollment has not been done previously, the attestation will fail.

The following diagram explains all exchanges between the libraries, device, and attestation server during an attestation request:



8 Prerequisites

8.1 Baseline Platforms

The authorized platforms to deploy the application must:

- embed the Android version 13 android api 33 or above.
- support and hardware backed key storage (KeyStore) for the Android platform like TEE or Strongbox.

8.2 Permissions

Our SoftPOS library activate following android permissions:

- ACCESS_NETWORK_STATE: This is used to check if the device is connected to a network
- INTERNET: This is used for the enrollment and attestation requests
- READ_EXTERNAL_STORAGE: This is used to read the external storage (for example, to read the certificate)

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

Our libraries are compiled for this minimal sdk version. For more details see the chapter *Baseline Platforms*.

8.3 ALCINEO SoftPOS libraries dependencies

Gradle is the preferred build system for working with the ALCINEO Android SoftPOS.

First of all, ALCINEO SoftPOS libraries (described in previous chapter) should be placed in your app/libs folder. Then, in your build.gradle, please add the following:

```
dependencies {
    implementation fileTree(include: ['*.aar', '*.jar'], dir: 'libs')
    ...
}
```

8.4 Application mandatory dependencies

The ALCINEO Android SoftPOS requires some other dependencies, under the previous dependency please add the followings:

```
dependencies {

    ...
    // ### SoftPOS required ###
    // ### dependencies below must be included in a SoftPOS project ###
    implementation 'com.google.guava:guava:$VERSION'
    implementation 'com.neovisionaries:nv-i18n:$VERSION'
    implementation 'org.jetbrains:annotations:$VERSION'
    implementation 'androidx.lifecycle:lifecycle-process:$VERSION'
    // Http
    implementation 'com.squareup.retrofit2:retrofit:$VERSION'
    implementation group: 'com.squareup.retrofit2', name: 'converter-
    ↪jackson', version: '$VERSION'
}
```

8.5 Assets

The SoftPOS payment libraries can configure the different kernels through terminal settings files. You **must** have a “settings” folder in your application assets folder which contains terminal settings files as follows:

- /assets/settings/
 - CA_Keys.bin
 - exception_file.bin
 - languages.bin
 - revocation_list.bin
 - TerminalSettings_mcl.bin
 - TerminalSettings_pw.bin
 - TerminalSettings_XX.bin - for each kernel in your solution

The terminal settings are copied at first installation.

Please refer to Terminal Settings documentation for more details.

9 SoftPOS initialisation setup

Warning: Steps to be implemented before the very first transaction

Our SDK includes a sample application, which we strongly encourage you to review in order to better understand how our libraries function.

Nevertheless, here are the essential steps you need to implement to integrate our libraries into your application.

Note: Although the documentation states that this step must be performed in your application before any transaction, that doesn't mean we advise developers to start the SDK integration with it. During the development phase, we provide a test attestation server that is pre-configured in the BonAppetit sample app. We therefore suggest keeping this default setup and focusing first on developing your own features.

9.1 Basic example

The sample code below provides a very basic implementation designed to give you an overview of the required sequence of steps.

The details of each step will be explained afterward.

```
// Initialization of the libraries
AttestationAgent.init(context, "https://CSR_HOSTNAME:PORT", "https://A&
→M_HOSTNAME:PORT", 60000);

SecurityAPI.initialize(activity);

SoftposAPI.initialize(activity);

PinpadAPI.initialize(context);

// Update the initial session key
try {
    UpdateInitialSessionKeyAPI.updateInitSessionKey();
}
// Enrollment is mandatory
catch (AttestationRequestException ex) {
    if (ex != null && ex.getAttestationResponseCodeEnum() ==
→AttestationResponseCodeEnum.SERVER_DUID_NOT_ENROLLED) {
        EnrollmentAPI.newEnrollment();
    }
}
```

(continues on next page)

(continued from previous page)

```

    }
}
// TR34 IMEAK injection
RkiExample.tr34IMEAKInjection();

```

9.2 Libraries initialization

You need to call one API to register the URL for the attestation server. The API is:

where

- “https://CSR_HOSTNAME:PORT” URL of server for CSR Request
- “https://A&M_HOSTNAME:PORT” URL of server for Attestation and Monitoring
- timeout_in_ms is the timeout for the initialization in milliseconds.

Then the ALCINEO Android Security library must be initialized with the following instruction:

```
SecurityAPI.initialize(activity, listener);
```

Note: This security library initialization must be done **before** the SoftPOS SDK initialization.

The library initialization will:

- Fetch device information for the attestation server.
- Create services for key injection, attestation and audit log.
- Start a security system check routine (running every second, monitoring Developer mode status and camera usage during a transaction). ****If this check fails, the SDK will cancel the transaction, close all services and the payment library (calling *SecurityAPI.terminate()*), stop the security check routine, and notify the application (via the listener) of the error.**

The ALCINEO SoftposAPI library must also be initialized with the following instruction:

```
SoftposAPI.initialize(activity);
```

The library initialization will:

- Create the main communication service used by the payment library.
- Create all payment relative services like Kernel administration or transaction manager.

Then ALCINEO PinpadAPI library must also be initialized with the following instruction:

```
PinpadAPI.initialize(context);
```

9.3 Update Initial Session Key

The attestation protocol defined by Alcineo specifies the transmission of a shared key **INIT_SES** during the enrollment process.

This key is used to secure the communication protocol between the A&M server and the device.

The enrollment procedure will be described shortly.

However, prior to initiating the enrollment, it is required to initialize an INIT_SES key as follows:

```
UpdateInitialSessionKeyAPI.updateInitSessionKey();
```

Because enrollment should be performed only once (after the first installation of the application), the updateInitSessionKey method throws an exception if the device has not yet been enrolled.

Catching that exception lets you know that you must run the enrollment sequence.

Note: the *Update Initial Session Key* command must be performed at each new production delivery.

9.4 Enrollment

Only an enrolled device can perform a transaction. No payment transaction can be performed if the enrollment step has not been completed.

If the device attempts to start a transaction without being enrolled, it will receive an error from the Attestation Server.

To start an enrollment please call:

```
EnrollmentAPI.newEnrollment();
```

The device will be fully enrolled on the server side once it has successfully sent a new command using the INIT_SES key. This command is considered as an acknowledgment by the server.

9.5 TR34 Remote Key Injection

9.5.1 Purpose

The SoftPOS solution uses DUKPT keys to encrypt and sign payment data and PIN blocks.

Note: For the sake of clarity, we will refer to the IMEAK in this section, but please note that the exact same principles apply to the IPEK

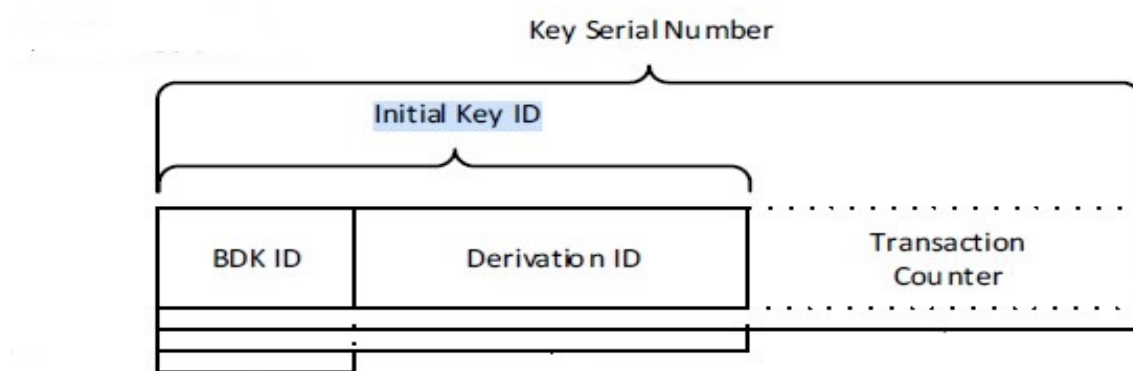
The customer's backend system is in charge of updating the DUKPT keys in the device via the TR34 key injection interface.

It must respect two constraints to meet security standards:

- The initial DUKPT keys injected must be different for each COTS device.
- A key that was previously installed must not be installed again.

This guarantees that a given session key will be used only once on a unique device.

To ensure this, we advise incrementing the *Initial Key ID* (as defined in the X9.24 standard) with each IMEAK generation by the EMV backend system (and similarly for IPEK if a PIN pad is present).



As stated in ANSI X9.24-3-2017 §5.3:

“Initial Key IDs shall NOT be re-used, even with the same device, as this would result in re-using the same Initial Key.”

To initiate the process you need to inject an initial DUKPT key (called **IMEAK**) into the SoftPOS library.

Afterwards, the DUKPT key **MEAK** is derived from the initial key. Moreover, the MEAK is diversified into two keys:

1. **MEK** – used to encrypt transaction data
2. **MAK** – used to authenticate (sign) transaction data

Note: The application developer can obtain the KSN of the current keys installed via the *getKSN()* interface.

For more details on the DUKPT specification, see *ANSI X9.24 Part 3 (References [1])*.

To inject the initial key, there are two possibilities:

1. **option 1** : inject a transport key in TR34 format (the transport key for this is called **TR_IMEAK**), then inject the IMEAK in TR31 format protected with that TR_IMEAK.
2. **option 2** : Inject the IMEAK directly in TR34 format

For more details about all keys, refer to the [Key details](#) section.

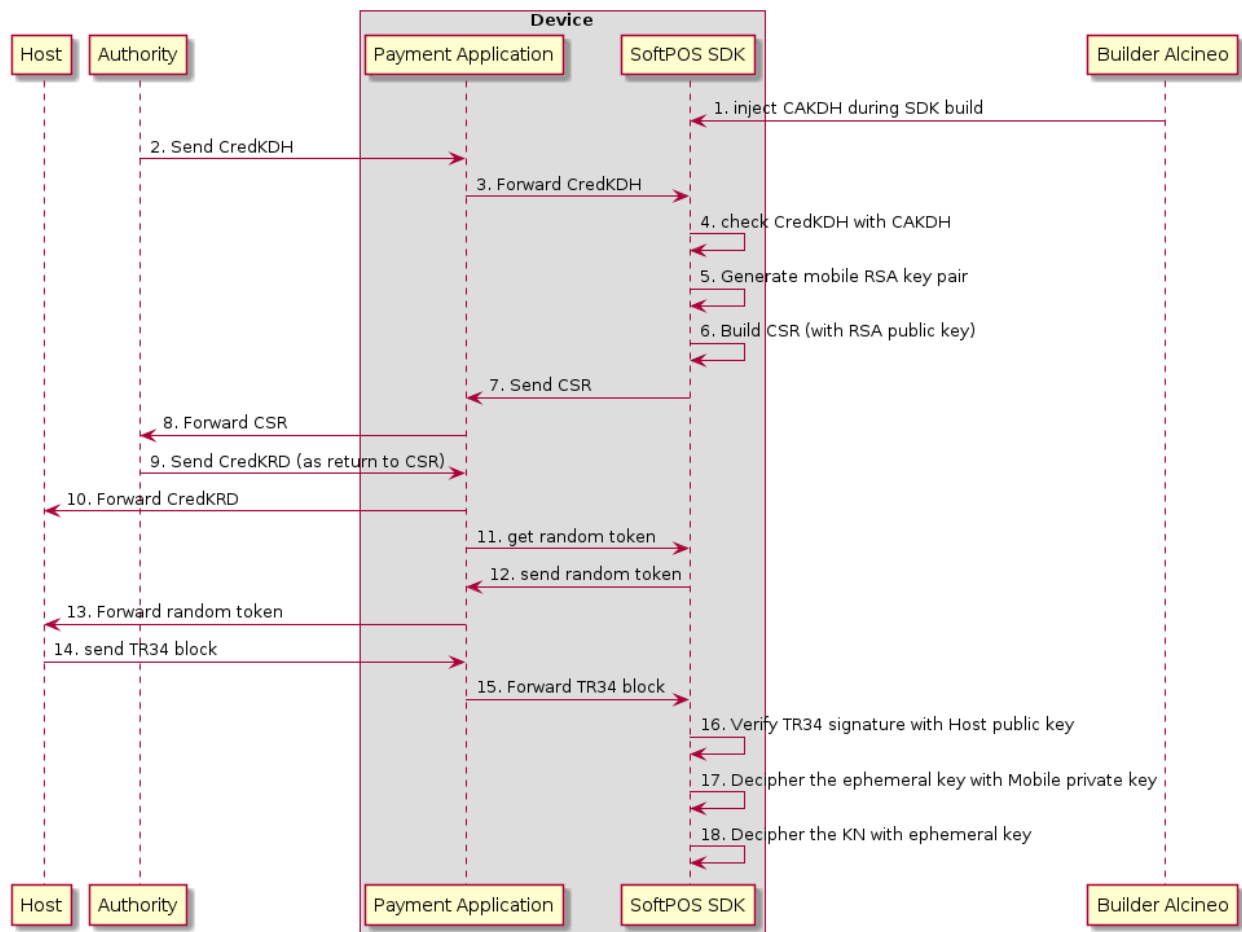
When the initial DUKPT key is injected into the device, new DUKPT session keys are generated and securely stored in the device. Next, the initial DUKPT key is permanently deleted. At this step, the current Key Serial Number (KSN) has a counter value of 0.

At each new transaction, a new session key is generated while the old one is deleted. Once a session key is used, it is deleted from the device. A new DUKPT key is generated and replaces the old one in secure storage, and then the current KSN counter is incremented.

9.5.2 TR34 Description

9.5.2.1 Flow

TR34 defines a key-block format for transporting symmetric keys.



9.5.2.2 TR34 recommended CA scheme

The TR34 root CA does not belong to Alcineo; it is issued by another stakeholder in the payment chain, typically the payment-backend operator.

For example the following CA hierarchy:

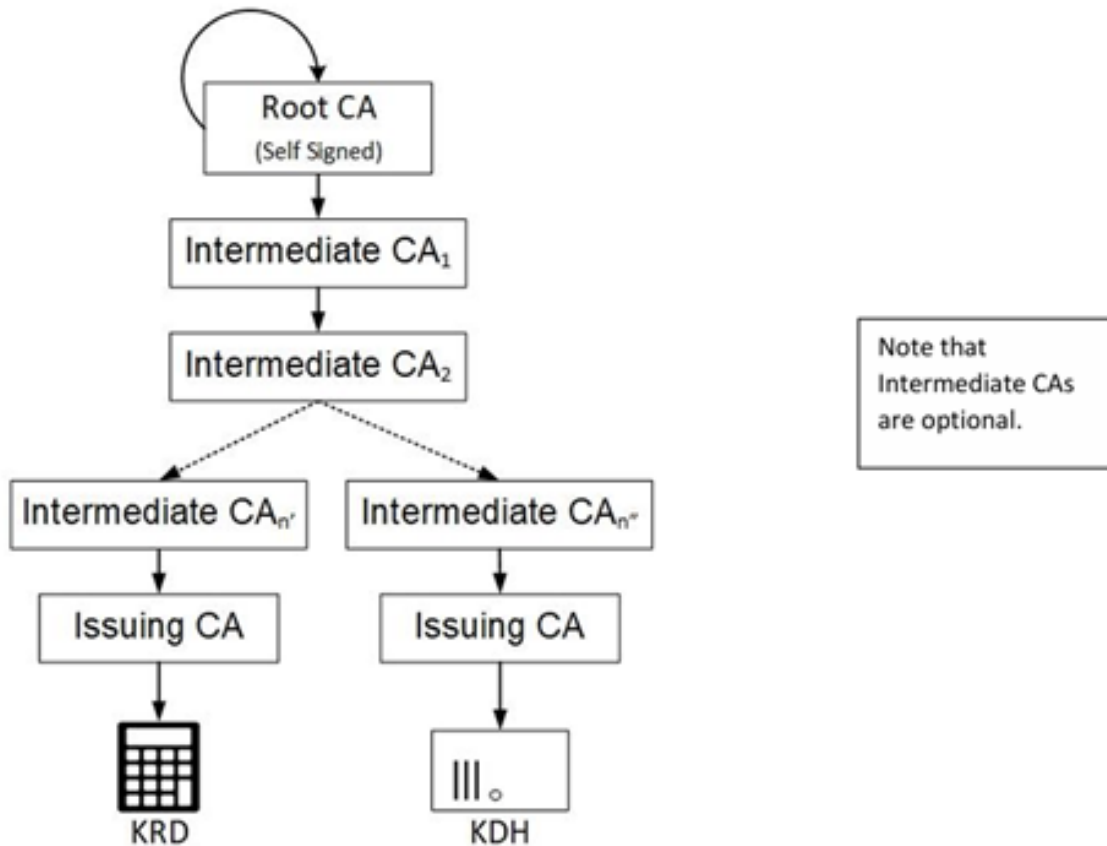


Figure 2

9.5.2.3 TR-34 replay prevention

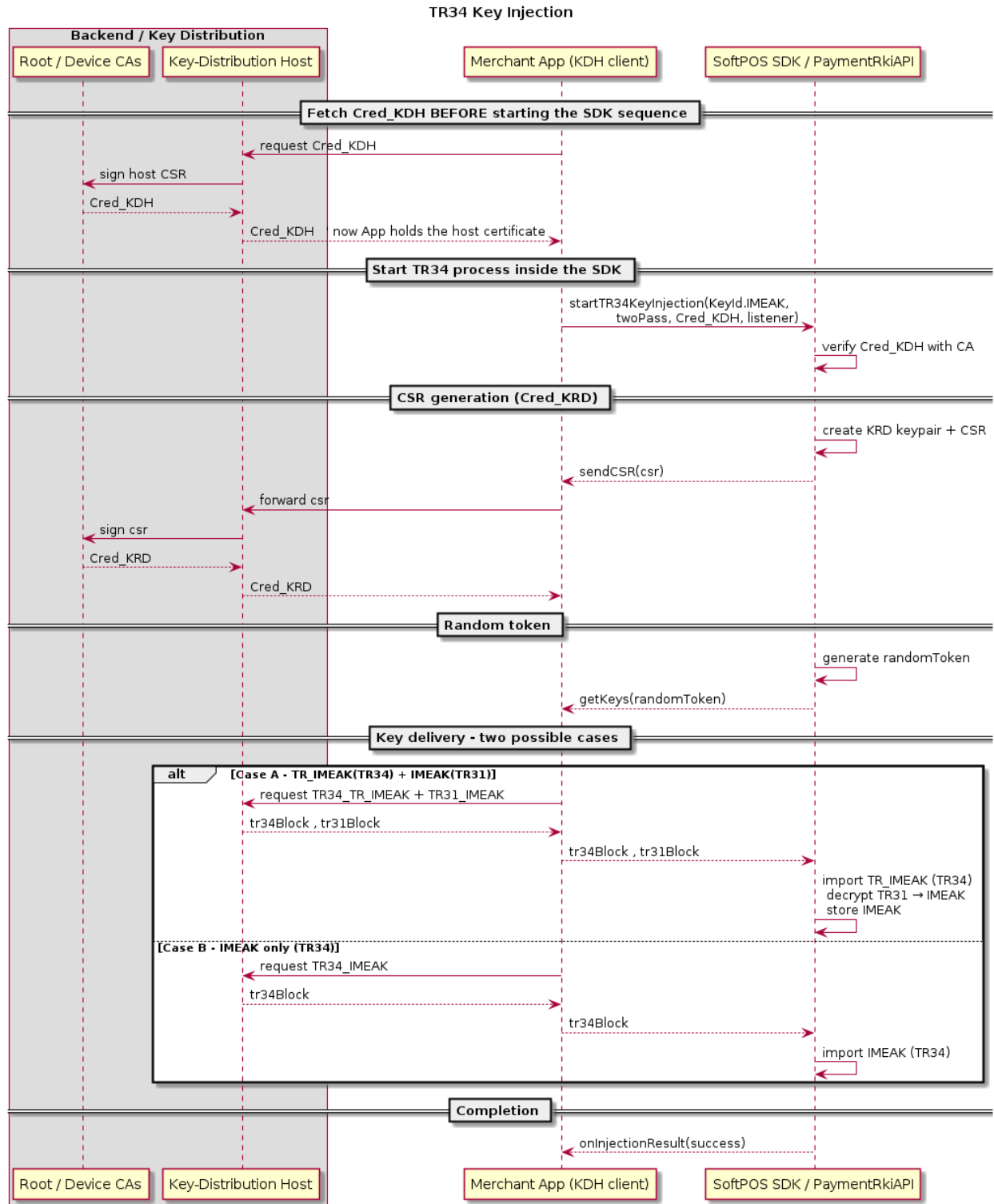
TR34 offers **two distinct protocols** to prevent replay attacks, chosen according to whether the *Key Distribution Host* (KDH) and the *Key Receiving Device* (KRD) can exchange messages in real time:

- **Two-pass protocol** – recommended when the KDH and KRD *do* have real-time connectivity.
- **One-pass protocol** – permitted only when the KDH and KRD *cannot* communicate in real time.

Because the Alcineo SoftPOS deployment operates in a **real-time** environment, the solution **must implement the two-pass protocol**.

9.5.3 TR34 API

9.5.3.1 Call sequence



9.5.3.2 API for option 1

```
byte[] credKdh = ... // array of Cred_KDH bytes; this buffer is an X.
    ↳509 certificate (signed by CA KDH, which is injected into the SDK
    ↳during production build)
boolean twopass = true; // true for TR34 two-pass process, false for
    ↳one-pass
PaymentRkiAPI.startTR34KeyInjection(KeyId.IMEAK, twopass, credKdh,
    new InjectionTR34_TR31Listener() {

        @Override
        public void sendCSR(byte[] krdCSR) {
            // send CSR to authority to generate Cred_KRD and send it
    ↳to KDH
        }

        @Override
        public TR34_TR31_keys getKeys(byte[] randomToken) {
            byte[] tr34Block = ... // get TR34 block from KDH
    ↳containing TR_IMEAK
            byte[] tr31Block = ... // get TR31 block from KDH
    ↳containing IMEAK
            return new TR34_TR31_keys(tr34Block, tr31Block);
        }
    });
```

9.5.3.3 API for option 2

```
byte[] credKdh = ... // array of Cred_KDH bytes; this buffer is an X.
    ↳509 certificate (signed by CA KDH, injected into SDK during
    ↳production)
boolean twopass = true; // true for TR34 two-pass process, false for
    ↳one-pass
PaymentRkiAPI.startTR34KeyInjection(KeyId.IMEAK, twopass, credKdh,
    new InjectionTR34_TR31Listener() {
        @Override
        public void sendCSR(byte[] krdCSR) {
            // send CSR to authority to generate Cred_KRD and send it
    ↳to KDH
        }

        @Override
        public TR34_key getKeys(byte[] randomToken) {
            byte[] tr34Block = ... // get TR34 block from KDH
    ↳containing IMEAK
        }
    });
```

(continues on next page)

(continued from previous page)

```

        return new TR34_key(tr34Block);
    }
}
};

```

The key in option 2 is provided entirely in TR34 format. (Refer to the appropriate documentation for details on constructing TR34 key blocks.)

9.5.4 Key Bundle Format

As previously we will detail here only IMEAK bundle key but this is the same for IPEK.

The format of the keys to be injected in the device is TR34. A TR34 key block contains the key to inject (either TR_IMEAK or IMEAK).

This key is protected by an ephemeral AES-128 key called **EPH_TR_IMEAK** (or **EPH_IMEAK** for IMEAK).

This ephemeral key is enciphered by **cred_KRD** (RSA 3072).

- The **cred_KDH** (host public key certificate) is provided to the SoftPOS SDK during the first call to *startTR34KeyInjection*. This cred_KDH is verified at key injection time using the **CA_KDH** (the certificate authority for KDH) which is embedded in the SoftPOS SDK at build time.
- The **cred_KRD** (device key certificate) is generated for each key injection. You need to send its CSR to your backend server/CA to sign it and inject the resulting certificate into your HSM.

An initial DUKPT key is composed of two parts: the key data and its Key Serial Number (KSN).

Below are some details of the KSN transmission.

The ALCINEO SoftPOS solution follows these rules regarding KSN transmission:

- The KSN is included in the optional data field of the TR31 key block (preceded by the identifier “KS”).
- The KSN is written in full, with its counter value set to 0.

For more details on the TR34 standard, see *ANSI X9 TR34-2012* ([References](#) [4]).

For more details on the TR31 standard, see *ANSI X9 TR31-2018* ([References](#) [2]).

9.5.5 Example of TR31 Key Generation After TR34 TR_IMEAK Injection

This case corresponds to the option 1 of the key injection.

In the delivered SDK (development phase), a test key is already present to allow developers to test the key injection feature.

TR_IMEAK = 0x55555555555555555555555555555555

This transport key is the **KbPK** as defined in *ANSI X9 TR31-2018* ([References](#) [2]) for TR31.

This key is used for derivation with the AES-128 algorithm.

TR_IMEAK is used to generate the encryption key and authentication key of the TR31 key (as described in *ANSI X9.24-3-2017* ([References](#) [2]), Figure 6 – *Deriving an AES-128 encryption and authentication key*).

In the following example, the random value chosen for padding is set to “00”.

The initial key injected is:

```
0x1436692E6A221C29A1EC1B62EB315D5B
```

Understand that this initial key is chosen arbitrarily (for example purposes).

It does not come from a BDK derivation.

In your environment, when you generate your own BDK, the initial key will appear as a random number.

The format of the resulting TR31 key is as follows:

```
// Header
D0144B1AX02N0200KS1C123456789012345600000001PB04

// with optional block ("KS" || length || KSN) + padding (4 characters)
KS1C123456789012345600000001PB04

// Cipher Text
37ED8C981331081D6A088B89CAD53194766A8C842D73080E52D2EA86FEDB5F91

// MAC (16 bytes)
02FF0DE7A7CFA9A58E0A8F221A15CF2B
```

The TR31 key is the concatenation of the Header, Cipher Text, and MAC.

9.5.6 Example of TR31 Key Block with TR34 IMEAK Injection

This case corresponds to the option 2 of the key injection.

In the delivered SDK (development phase), an ephemeral key (EPH_IMEAK) is used to protect the TR31 included in the TR34 block.

This ephemeral key correspond to the **KbPK** as defined in *ANSI X9 TR31-2018* ([References](#) [2]) for TR31. This key is used for derivation with AES-128.

EPH_IMEAK generates the encryption key and authentication key of the TR31 key (as described in *ANSI X9 TR31-2018* ([References](#) [2]), Figure 6).

Again, in this example the random padding is “00”.

The initial key injected is:

```
0x1436692E6A221C29A1EC1B62EB315D5B
```

As noted, this initial key is chosen arbitrarily (not derived from a BDK). When you generate your own BDK, the initial key will be a random number.

The format of the TR31 key is as follows:

```
// Header
D0144B1AX02N0200KS1C123456789012345600000001PB04

// with optional block ("KS" || length || KSN) + padding (4 characters)
KS1C123456789012345600000001PB04

// Clear Text
1436692E6A221C29A1EC1B62EB315D5B
```

The TR31 key block included in the TR34 format has its header in clear text (as shown above).

10 Performing a transaction

When the SoftPOS libraries are initialized, key injection is done, and the enrollment is completed, the application can start a transaction.

Note: All transaction API can be found under the package `com.alcineo.softpos.payment.api...`

API are organized by category, for example: all API relative to a `transactionParameters` can be found in `TransactionAPI` class.

10.1 Start transaction

The transaction is started with the method `startTransaction` of the class `TransactionAPI`.

Note: An example of implementation is available on sample “BonAppetit” cf. `ShoppingFragment.java`, function `onStartTransactionClick`.

```
TransactionAPI.startTransaction( TransactionParameters,
    ↳ transactionParameters,
                                TransactionEventListener,
    ↳ transactionEventListener,
                                PinProcessEventListener,
    ↳ pinEventListener,
                                LifecycleOwner lifecycleOwner)
```

- The first object is `TransactionParameters`, this object is used to configure the payment transaction.
- The second object is the `TransactionEventListener` interface that handles the POS user interface events and the transaction: transaction transmitted online and transaction completed. The application class that uses the `TransactionAPI()` class can extend this interface and use “this” for the second parameter of the `startTransaction` method.
- The third object is the `PinProcessEventListener` interface that handles the PIN process events. The application class that uses the `TransactionAPI()` class can extend this interface and use “this” for the third parameter of the `startTransaction` method.
- The fourth `LifeCycleOwner` object represents the lifecycle of the Android application activity. It is used by the SoftPOS library to capture Android lifecycle events. For example if the application goes in background, `Lifecycleowner` will handle it. As SoftPOS library needs to know the status of the Android application when transaction is active, these parameters need to be set to the fragment/activity where the transaction was launched. see {[@link https://developer.android.com/topic/libraries/architecture/lifecycle#lco](https://developer.android.com/topic/libraries/architecture/lifecycle#lco)} for more.

10.1.1 TransactionParameters

The TransactionParameters object is used to configure the transaction. It contains the following parameters:

```
private final int      cardInteractionWaitingTime      //┐
↳The transactionParameters card interaction waiting card
private final BigDecimal amount;                      //┐
↳The transactionParameters amount
private final BigDecimal amountOther;                 //┐
↳The transactionParameters refund amount
private final BigDecimal balanceBefore;               //┐
↳The balance before the transactionParameters
private final BigDecimal balanceAfter;                //┐
↳The balance after the transactionParameters
private final Byte     transactionType                 //┐
↳The transaction type
private final CurrencyCode currencyCode;              //┐
↳The currency used to process transactionParameters
private final String    categoryCode;                 //┐
↳The category code
private final String    merchantData;                 //┐
↳The merchant data
```

For example in the BonAppetit sample application, the transactionParameters are set as follows:

```
final int cardInteractionWaitingTime = 15;
final BigDecimal transactionRefund = null;
final BigDecimal balanceBefore = null;
final BigDecimal balanceAfter = null;
final TransactionType transactionType = TransactionType.PURCHASE;
final String transactionCategoryCode = "F";
final String transactionMerchantData = StringUtils.convertBytesToHex(
↳"Alcineo".getBytes(StandardCharsets.UTF_8));
final CurrencyCode transactionCurrency = mShoppingViewModel.
↳transactionCurrencyCode.getValue();
final BigDecimal transactionAmount = mShoppingViewModel.
↳transactionAmount.getValue();
```

10.2 Transaction (and Pin) Events

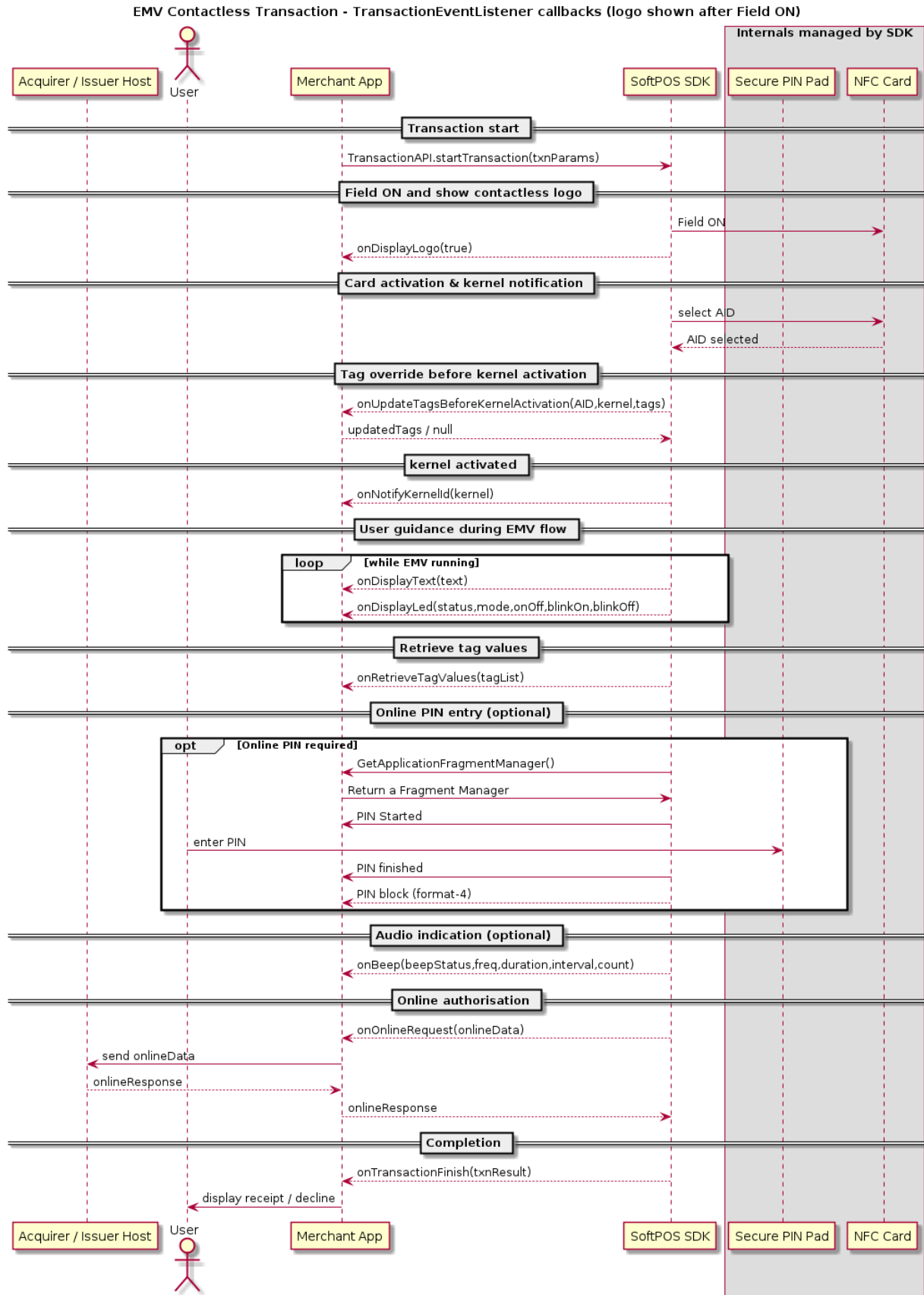
During the transaction, the application will receive several events.

The following diagram shows the sequence of events and their order.

The transaction events are defined in the interface TransactionEventListener and the events

relative to PIN entry are defined in `PinProcessEventListener`. The application must implement these interfaces to handle the events.

Note: An example of implementation is available on sample “BonAppetit” cf. `TransactionViewModel.java`. `TransactionViewModel` implements the interface `TransactionEventListener` and `PinProcessEventListener`.



11 Sensitive Data Management

11.1 Smart Card Sensitive Data

Sensitive data in this context refers to assets coming from the smart card. They are defined as follows:

Cardholder Data includes: - Primary Account Number (PAN) - Cardholder Name - Expiration Date - Service Code

Sensitive Authentication Data includes: - Full track data (magnetic-stripe data or equivalent data on a chip) - CAV2/CVC2/CVV2/CID - PINs/PIN blocks

All sensitive data must not be handled by the Java code in plain text. Only their encrypted form should be passed through the Java layers on the device.

11.2 Sensitive Data Encryption

At the end of a transaction, two configuration tags (0xDFDF02 and 0xDFDF03) allow you to specify which data tags will be output by the terminal at the end of the transaction or during an online request.

- Tag **0xDFDF02** defines the data elements (tags) for an EMV-type transaction output.

(Note: By default, the data specified by these tags would be output in plain text.)

To secure the sensitive data as described above, their output must be encrypted. For this purpose, the terminal settings include a tag **0xDFDF33** which specifies which tags are sent in encrypted form. The device will assemble all those specified tags into a buffer and encrypt it.

Then, during the online request or at the end of the transaction, the Java application receives:

- Tag **0xFF8105**: contains the clear data as defined by tags 0xDFDF02 and 0xDFDF03.
- Tag **0x74**: contains encrypted data and is composed as follows:
 - tag **0xDFDF50** – the KSN used for the encryption
 - tag **0xDFDF51** – the encrypted data, encrypted with the DUKPT keys (AES-128 in CBC mode with PKCS#7 padding)
 - tag **0xDFDF53** – the initial IV value used for the CBC mode encryption
- Tag **0x75**: contains the signature and is composed as follows:
 - tag **0xDFDF50** – the KSN used for the signature
 - tag **0xDFDF52** – the signature result, generated using the DUKPT keys with the CMAC algorithm. (The signature is calculated over the clear data included in tag 74, before that data is encrypted.)

All these tags and their values are defined in the terminal settings file/binary. The application developers are responsible for configuring the terminal settings correctly to ensure sensitive data is protected.

11.3 Outcome Example

In the BonAppetit sample code, a log trace is added to output the encrypted data. It is possible to capture it with the *logcat* command. Below is a method to decrypt this data and check the signature. Let's suppose the trace output is:

```
encrypted data = TlvItem(tag=TlvTag(74),
    ↪value=DFDF500C123456789012345600000005DFDF51820030DE66E229DE8AC2124
    ↪3F570B194D588C02DA9F34D9F8A4A21C81E2AF9E0C68A17CD0C83DE32E50BF59F5
    3FC5C8E73AA4CDFDF5310463FEE6B8B1284C26FF0121FB0F1ED7C)
signed data = TlvItem(tag=TlvTag(75),
    value=DFDF500C123456789012345600000005DFDF5210FAAD3DECF
    B238F8D916917D47145B64F)
```

11.3.1 Encrypted data

The encrypted data (tag **0x74**) is provided in TLV format.

The tag **0xDFDF50** gives the KSN used for the encryption (the KSN is 12 bytes).

For example: **0xDFDF50** 0C 123456789012345600000005

The tag **0xDFDF51** gives the encrypted data. This data is encrypted with the DUKPT keys in CBC mode with PKCS#7 padding.

For example: **0xDFDF51** 820030 DE66E229DE8AC21243F570B194D588C02DA9F34D9F8A4A21C81E2AF9E0C68A17CD0C83DE32E50BF59F53FC5C8E73AA4C

The tag **0xDFDF53** gives the initial vector used for the CBC mode encryption.

For example: **0xDFDF53** 10 463FEE6B8B1284C26FF0121FB0F1ED7C

11.3.2 Signature

The signature is generated over the data in plain text. The signature (tag **0x75**) is also in TLV format.

Again, the tag **0xDFDF50** gives the KSN used for the signature: **0xDFDF50** 0C 123456789012345600000005

The tag **0xDFDF52** gives the signature result, produced with the DUKPT keys (CMAC algorithm): **0xDFDF52** 10 FAAD3DECFB238F8D916917D47145B64F

11.3.3 DUKPT Keys Diversification

In our example, the initial DUKPT key (IMEAK) has the value:

IMEAK = 0x1436692E6A221C29A1EC1B62EB315D5B

To obtain the session DUKPT key used to encrypt the data and to sign them, we need to diversify the IMEAK.

The key diversification is performed with the following parameters:

- DUKPT algorithm AES
- Input key designation: IK = IMEAK
- Working key type: AES-128
- KSN set to 1234567890123456000000005

The counter value of the session key is given at the end of the KSN, in our example the value is 4.

The key diversification and key variant computations return the key MAK to sign

usage = message authentication generation:

0x00B6598943459390C0914918233FA9F5

and the key MEK to encrypt

usage = data encryption encrypt:

0x26B2D75EC7571ADC7605359FCDD96955

11.3.4 Data decryption

To decrypt the encrypted data:

0xDE66E229DE8AC21243F570B194D588C02DA9F34D9F8A4A21C81E2AF9E0C68A17CD0
C83DE32E50BF59F53FC5C8E73AA4C

use the session key MEK:

0x26B2D75EC7571ADC7605359FCDD96955

and the algorithm **AES 128** in **CBC** mode with IV =
0x2E215B70A51D8B359AC7D1A4AD2639A4.

The decrypted data is equal to:

0x57135134148633967356D25012016369673060000F5A0851341486339673565F2403
2501310B0B0B0B0B0B0B0B0B0B

The final data 0B0B0B0B0B0B0B0B0B0B0B0B does not come from the card, this is the PKCS7 padding format.

11.3.5 Signature verification

To verify the signature of the plain-text data:

0x57135134148633967356D25012016369673060000F5A0851341486339673565F2403250131

Please use the session key MAK:

0x00B6598943459390C0914918233FA9F5

and the algorithm **CMAC AES**.

The result is 0xFAAD3DECFB238F8D916917D47145B64F, the same as the one in the tag 0xDFDF52. So the signature is correct.

12 PIN Management

12.1 Introduction

Unlike earlier releases of the Alcineo SDK, the PIN-pad is now handled entirely inside the SDK.

The SDK itself decides when to display the PIN-pad, based on the payment-kernel's instructions.

Only two events are reported back to the application, as described in the **`Transaction Events`** chapter.

Consequently, this chapter no longer deals with integrating a PIN-pad UI; instead, it focuses on how to retrieve the resulting PIN-block data.

Likewise, the injection of the DUKPT keys used to encrypt the PIN is not covered in this chapter, because the procedure is identical to the one already described for encrypting payment data.

12.2 PIN encryption

12.2.1 ISO 9564-1 Format 4

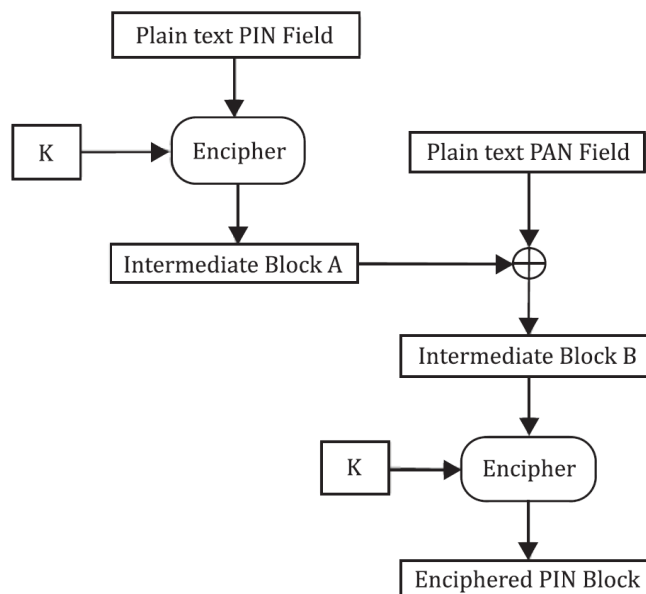
An important security requirement in the SoftPOS context is that the PAN and the PIN must stay segregated while they are present in the mobile.

In the other hand, in the standard ISO 9564-1 amendment 1, the PIN block is formatted with the PIN and the PAN.

More precisely, the format 4 block generation is performed with two encryption processes.

First a PIN block is enciphered. Then this encryption result is Xored with the PAN.

Finally the last result is encrypted as the scheme below.



In the Alcineo's solution the plain-text PIN field data is formatted in 16 bytes included its length size and padding following ISO 9564 standard and then enciphered with the AES PEK DUKPT key.

The result is the intermediate block A.

Then the intermediate block A is xored with the PAN.

Finally, the block is enciphered with the same DUKPT key to finalize the format 4.

This constant PAN is used to avoid to mixed the real PAN with the PIN.

The final result is provided to the EMV back-end with the KSN of the current PEK during the online request.

The ENV back-end system is in charge to continue the online PIN processing. This step is out of scope of the Alcinéo solution.

12.2.2 PIN decryption example

The iso 9564-1 amendment 1 for format 4, describe how to build a PIN block including with the PAN in the encryption.

The PAN is transmitted in the tag 5A by the card or included in the tag 57.

1. Retrieved tag 0x99 in the online request contains this data. We will illustrate this example with following value :

0x991E10E05FA0FADBFA95F46C57FF53E05330E30C123456789012345600000002

- The structure is the following.
 - 0x99 : tag for PIN
 - 0x1E : length of whole block
 - 0x10 : length of PIN block
 - 0xE05FA0FADBFA95F46C57FF53E05330E3 : Enciphered PIN Block in format 4
 - 0x0C : KSN length
 - 0x123456789012345600000002 : KSN

2. Retrieved Key injected

- The IPEK test injected is 0x1436692E6A221C29A1EC1B62EB315D5B.
- KSN counter is equal to 2. So after calculating the second session key for PIN with DUKPT 2017 algorithm, we find:

PIN Encryption Key = 0x4832E218343F264904BFD20C28C47685.

3. Uncipher Enciphered PIN Block received

- Uncipher the Enciphered PIN Block with PIN Encryption Key with **AES 128** in ECB mode, the result is;

Intermediate Block B = 0x71E336A8070C2045C69C91D0C55C78C6.

4. Retrieved PAN

- In online value retrieved tag 74, in this example value of tag 74 is

DFDF500C123456789012345600000005DFDF51820030DE66E229DE8AC21243F570B194D588C02DA9F34D9F8A4A21C81E2AF9E0C68A17CD0C83DE32E50BF59F53FC5C8E73AA4CDFDF5310463FEE6B8B1284C26FF0121FB0F1ED7C

- The tag 0xDFDF51 is equal to:

DE66E229DE8AC21243F570B194D588C02DA9F34D9F8A4A21C81E2AF9E0C68A17CD0C83DE32E50BF59F53FC5C8E73AA4C

- The tag 0xDFDF53 which is used as IV is equal to:

463FEE6B8B1284C26FF0121FB0F1ED7C

- The KSN (tag 0xDFDF50) is equal to : 1234567890123456000000005

- As the IMEAK = 0x1436692E6A221C29A1EC1B62EB315D5B

- The MEK used to encipher the payment data following the DUKPT specification is: 26B2D75EC7571ADC7605359FCDD96955

- Let decipher in CBC mode with IV = 463FEE6B8B1284C26FF0121FB0F1ED7C the tag 0xDFDF53

with this key MEK, we obtain:

57135134148633967356D25012016369673060000F5A085134148633967356
4032501310B0B0B0B0B0B0B0B0B0B

- The tag 5A is present in the deciphered data (TLV format):

5A 08 5134148633967356

It has 16 digits length.

- Sometime the tag 5A is not present in deciphered data. In this case, the PAN value can be retrieved from the beginning of tag 57 up to the “D” separator.

57 13 5134148633967356 D 25012016369673060000F

5. We can continue the PIN block deciphering. Construct the Plain Text PAN Field following the format 4:

Plain test PAN Field = 0x45134148633967356000000000000000

The first digit means that the PAN length has 4 digits beyond to 12 digits.

Remark: if the PAN was:

5A 08 374245001781003F then , as the F caractere is padding,

plain test PAN Field will be = 0x33742450017810030000000000000000

6. Xoring Intermediate Block B and Plain test PAN Field give Intermediate Block A

0x71E336A8070C2045C69C91D0C55C78C6

xor

```
0x45134148633967356000000000000000
```

Intermediate Block A = 0x34F077E064354770A69C91D0C55C78C6.

- ## 7. Deciphering Intermediate Block A with PIN Encryption Key

PIN Encryption Key = 0x4832E218343F264904BFD20C28C47685 (same at step 2)

Plain text PIN Field = 441414AAAAAAAAAA5FAF1DFB7E038B2C

As described in iso 9564-1 amendment 1 for format 4

- 0x44 : the first digit gives the format number / the second digit gives the PIN length
- PIN entered “1414” is in byte 2 and 3
- 0xAAAAAAAA are the fill digits (to allow PIN code up to 12 digits)
- 0x5FAF1DFB7E038B2C are the random digits

13 Audit Logs

13.1 Description

SoftPOS solution writes audit logs in the mobile during the application execution.

These audit logs are a part of the server monitoring.

The A&M server analyses these audit logs and checks that the device behaviour is correct.

As the SDK , the application can write audit log messages.

An example is given in the file *AuditLogExample.java*.

To be compatible with Alcineo format do not include in your message the comma ',' caractere nor the tidle '~'.

The server identifies them as separators.

All these audit logs are stored in the same file in:

```
/data/data/com.alcineo.bonappetit/files/auditlog.txt
```

13.2 Signature

The audit log file owns a signature compute with SIGN_AUDIT_LOG key to authenticate it.

The SoftPOS library gives acces to an interface to the application to test securely the audit log signature.

```
boolean isCorrectSignature = AuditLogAPI.isCorrectAuditLogSignatures();
```

In this case, the whole file authenticity is checked.

This check needs computation time proportionally to the audit log size.

If it is big, this could take several seconds.

The application developer will have to call this interface at the appropriate time, for example during the application initialisation and stop.

If the signature is wrong, the problem is logged in the audit file each time the interface is called.

Moreover, the signature of the audit log file is checked by the SDK each time a new command is executed.

13.3 Send of Audit Logs

13.3.1 The SDK

The SoftPOS SDK is in charge to send regularly the audit logs to the A&M Server, at least every 24 hours.

The default hours for sending audit log file is 0h00, but this hours is configurable by the application with following api:

```
AuditLogAPI.setHoursForSendAuditLog(int hour, int minutes, int seconds);
```

After preferences hours register, the next audit log files will be send 24h after the last file sent or at next preferences hours if this date/time is before next 24h hours.

13.3.2 The application

The application is in charge to send the audit logs at each application start and stop.

It can do it with following api:

```
AuditLogAPI.sendAuditLogToAttestationServer();
```

This interface allows also to the application to send the audit log more often then every 24 hours, if needed.

Once the audit log is sent correctly to the A&M server, the audit log is flushed automatically in the mobile and a new one is created including the message “The audit log file has been flushed”.

13.3.3 The A&M server

The A&M server is in charge to check the audit log signature and to analyse the file contents, see below.

The application manager has access to the audit log contents through the A&M server front end. For more details see the document Front End User Guide.

13.4 Audit logs Analysis

The A&M server is in charge to analyse the file contents to find eventual functional and security problems.

The audit log generation and analysis is a part of tamper detection mechanisms.

Below the different messages which are displayed in the audit logs.

This messages are already embedded in the SDK libraries.

The application developers have to complete this list with their own messages coming from their application.

Two levels of problems are possible : critical and medium.

A critical problem will induce a device blocking.

Instead, several medium problems are necessary to block the device.

When it is mentioned that the problem is critical, then the device must be blocked.

This means that the A&M server blacklists the mobile and refuses any new transaction with this device.

This is achieved also by sending an *incident command* from the application to the A&M server.

A critical problem induces also a deletion of the device keystores.

13.4.1 Routine messages

The routine messages are those written when the operations are executed correctly. They have the header `EVENT_SUCCESS`. This means that no security or functional problem has been detected.

- `SELF_TESTS,EVENT_SUCCESS,PINPAD`,Self tests are performed successfully,
- `SELF_TESTS,EVENT_SUCCESS,PAYMENT`,Self tests payment executed successfully,
- `AUDITLOG,EVENT_SUCCESS,PAYMENT`,Audit log file creation,
- `TRANSACTION_START, EVENT_SUCCESS, PAYMENT`,Start transaction,
- `ENROLLMENT,EVENT_SUCCESS,PAYMENT`,The enrollment request is sent successfully,
- `ENROLLMENT,EVENT_SUCCESS,PAYMENT`,The enrollment response is verified successfully,
- `KEY_INJECTION,EVENT_SUCCESS,PAYMENT`,Key injection is finished correctly,
- `KEY_INJECTION,EVENT_SUCCESS,PINPAD`,Key injection for PIN is performed successfully,
- `KEY_MANAGEMENT, EVENT_SUCCESS, PAYMENT`,New DUKPT key is generated,
- `KEY_MANAGEMENT, EVENT_SUCCESS, PINPAD`,A new DUKPT key for PIN is generated,
- `CIPHER_DATA, EVENT_SUCCESS, PAYMENT`,Payment data are enciphered and signed.

13.4.2 Selftests problems

If a problem occurs during a self test process, the application will not start correctly.

The cryptographic layers are not in functional state.

It is a critical problem. The A&M server system has to block this mobile for further analysis.

- SELF_TESTS, EVENT_FAILURE, PAYMENT, Self tests payment failed
- SELF_TESTS, EVENT_FAILURE, PINPAD, Self tests pinpad failed

13.4.3 Key management problems

The key management alerts are critical. The security of the mobile is compromised.

The payment data could not be deciphered correctly.

The customer backend system has to performed again a complete key injection process. Below, failure examples on key management.

- KEY_MANAGEMENT, EVENT_FAILURE, PAYMENT, Problem occurs during the new session key generation
- KEY_MANAGEMENT, EVENT_FAILURE, PINPAD, The PIN encipherment failed

13.4.4 Key injection problems

The key injection alerts is a medium. It could be due to a network problem. No data are compromised.

- KEY_INJECTION, EVENT_FAILURE, PAYMENT, Key injection is NOT finished correctly
- KEY_INJECTION, EVENT_FAILURE, PINPAD, Key injection for PIN failed

13.4.5 Attestation problems

If a problem occurs during an attestation process, a transaction can not start.

If this problem occurs repeatedly, the A&M server has to block this mobile for further analysis.

Below, failure examples on attestation and enrollment.

- ENROLLMENT, EVENT_FAILURE, PAYMENT, The enrollment response verification failed
- ATTESTATION, EVENT_FAILURE, PAYMENT, A problem occurred during the attestation request
- ATTESTATION, EVENT_FAILURE, PAYMENT, The attestation response verification failed
- ATTESTATION, EVENT_FAILURE, PINPAD, A problem occurred during the PIN attestation request

- ATTESTATION, EVENT_FAILURE, PINPAD, The PIN attestation response verification failed

13.4.6 Cipherring problems

If a problem occurs during the payment data, the enciphered payment data are not available for the backend system. The transaction finishes with “end application” status. This error is critical, the mobile must be blocked. If after a new key injection the error is yet present, the mobile stays blocked for further analysis.

- CIPHER_DATA, EVENT_FAILURE, PAYMENT, Problem occurs during the signature of payment data
- CIPHER_DATA, EVENT_FAILURE, PAYMENT, Problem occurs during the encipherment of payment data

13.4.7 Tamper problems

A tamper detection is a critical problem.

This error can happen when the mobile is rooted. The device will be blocked.

The two logs linked to these errors are the following.

- TAMPER, EVENT_FAILURE, PAYMENT, Tamper detection. The keystore is deleted.
- TAMPER, EVENT_FAILURE, PINPAD, Tamper detection. The keystore is deleted.

13.4.8 Signature problem

The following messages are written in the audit log function of type events:

when the current audit log has a bad signature,

- TAMPER, EVENT_FAILURE, PAYMENT, “Tamper detection - audit logs are corrupted - wrong signature !!!!
- TAMPER, EVENT_FAILURE, PAYMENT, “Tamper detection during check integrity - audit logs are corrupted !!!!

This failure can occur in the following cases:

- if an attacker has tried to change the audit log.
- a writing problem.
- if the FK_AUDIT_LOG key has been changed during the production phase.

So if a bad signature is detected, this does not guarantee that it is an attack.

This is why this problem is medium, and only repetitions of problem will induce a device blocking on the server side.

14 Error Management

During enrollment, attestation, or other commands, if anything goes wrong at the server level or at the SoftPOS level, the request will fail and the transaction will be aborted.

The application will receive an error code from the following list:

Value	Left byte	Right byte	Description
0x0000	Response received	No error	–
0x0001	Response received	Length error	The response data does not have the expected length
0x0002	Response received	Protocol Version error	The protocol version is not the expected one
0x0003	Response received	DUID error	The DUID is incorrect
0x0004	Response received	Production Version error	The Production Version is incorrect
0x0005	Response received	Decryption error	The SoftPOS library cannot decrypt the data
0x0006	Response received	Decrypted data error	The decrypted data does not match the data sent
0x0007	Response received	Signature error	The SoftPOS library cannot sign the data
0x0008	Response received	Wrong signature error	The computed signature is incorrect
0x0009	Response received	Empty response error	The attestation response is empty
0x000A	Response received	Invalid format error	The format of the attestation response is not valid
0x0FFF	Response received	Undefined error (client side)	Unknown error, even though the server returned a response

“Response received” indicates that a response was received from the server but analysis of that response led to the given error code.

Value	Left byte	Right byte	Description
0x1000	No response	Length error	The request data does not have the right length
0x1001	No response	Protocol Version error	This protocol version does not exist or is deprecated
0x1002	No response	DUID error	The format of the DUID is not correct
0x1003	No response	DUID not enrolled	The corresponding device is not enrolled
0x1004	No response	Production Version error	The production version is not the expected one
0x1005	No response	Decryption error	The attestation server cannot decrypt the data
0x1006	No response	Decrypted Protocol Version error	The decrypted protocol version differs from the clear-text version
0x1007	No response	Decrypted DUID error	The decrypted DUID differs from the clear-text DUID
0x1008	No response	Decrypted Production Version error	The decrypted Production Version differs from the clear-text version
0x1009	No response	AUID error	The AUID is not the expected value
0x100A	No response	Timestamp error	The timestamp is not in an acceptable range
0x100B	No response	Android Version error	The Android version is not on the whitelist
0x100C	No response	Security Patch error	The Security Patch level is not on the whitelist
0x100D	No response	Signature error	The attestation server cannot sign the data
0x100E	No response	Wrong signature error	The computed signature is incorrect

“No response” means the server did not send back a valid response, instead this error code was generated.

Other error codes may appear in server logs (not returned to the device) for post-treatment on audit logs:

Value	Left byte	Right byte	Description
0x2013	Not returned	AUDIT LOG BAD GLOBAL CHECKSUM	Logged when the audit log's global signature is invalid
0x2014	Not returned	AUDIT LOG NO CONTINUITY	Logged when a missing audit log is detected (gap in sequence)
0x2015	Not returned	KEYSTORE INTEGRITY FAIL	Logged at server startup if the keystore integrity check fails

Another case is the error codes on the TLS Client *Get* API: in that scenario, only an HTTP 400 *Bad Request* is sent back to the SDK without additional information. On the server side, a specific error code is logged with the following mapping:

Value	Left byte	Right byte	Description
0x1040	No response	Bad PKCS#10 error	Error with the CSR format received from SDK
0x1041	No response	Client Certificate Exception	Error when creating the TLS client certificate
0x1042	No response	Certificate Exception	Error signing certificate when creating the client certificate
0x1043	No response	Operator Creation Exception	Error during the client certificate creation
0x1044	No response	The CSR is refused by the server	The device already exists with a blocked or banned status
0x1045	No response	Bad CSR signature	The CSR signature does not match the server's signature
0x1046	No response	Client Certificate I/O Exception	Error on server side while storing the created client certificate

“Not returned” indicates the server logged this error during periodic processes not directly caused by a client request.

Value	Left byte	Right byte	Description
0x3001	No response	TRANSPORT ERROR	An error occurred during transport frame exchange

15 Libraries protection

This part explains what are the protections applied to the SoftPOS libraries.

15.1 DUKPT keys size

Legacy algorithms have a shelf life and may lose some of their effectiveness over time. Moreover, the solution must utilize the most robust and current encryption algorithms and key sizes to withstand new attacks and then ensure support into the future. This is why, it is highly recommended to use AES 128 bits as DUKPT keys.

An obfuscation and protection configuration is already applied to the ALCINEO Android SoftPOS libraries. There is no need to add any other specific obfuscation rules.

15.2 Active Security Checks

To ensure the software integrity, different protections are activated.

The anti-root detection: if the mobile is rooted then the code protection detects it at runtime and stops the application launching.

The anti-debug detection: if the debug session is executed on the code, the active protection detects it at runtime.

The integrity check: code protection method injects integrity checks at locations it considers to be both performance-insensitive and structurally safe.

If an integrity problem is detected the application is stopped.

15.3 Code Obfuscation

- Create junk code paths
- Create phony duplicates of existing code which never execute
- Add random unused jumps
- Unroll loops to generate more code
- Hide integers using opaque predicates
- Encrypt all string literals in the source code and inject code to decrypt them at run-time

15.4 White-boxes

The protected and obfuscated software allows also to embed secret keys and cryptography algorithms securely in white-boxes.

These white-boxes keys can not be revealed by an attacker even if he has access to the code.

They are protected against fault attack and side channel attacks.

It is not possible to analyze the cryptography algorithms used in the white-boxes.

The same algorithm executed twice will run through different path in the code due to the randomization method included in the code.

15.5 SSL Keys

The connection between the mobile application and the attestation server is secured with SSL protocol.

The mobile ensures that the server is the right one with verifying its certificate.

The key hierarchy is the following:

- the root certificate authority TLS_ROOT_CA
- the SoftPOS intermediate certificate authority TLS_SOFTPOS
- the server certificate TLS_SERVER

The CA root key and the intermediate certificate Hash for SSL pinning are hardcoded in the SoftPOS library.

A new SDK build is required when we change the certificate chain.

The SoftPOS Library has two CA root and two hashes hardcoded for key rotation purpose.

16 Security Guideline

This part provides a security guideline during development cycle of a Java code in SoftPOS project. As these guidelines are constantly evolving, the code needs regular updates to comply with new rules.

These guideline updates are done when:

- best practices are identified
- new security rules arise
- new attack path is known

The security rules are mandatory. They allow to avoid attack on Java application.

Not following them could compromise a security evaluation in a laboratory.

16.1 Platform baseline restriction

as mentionned previously, the SoftPOS solution is certified for android version greater than 13. Please ensure that minSDK is greater than 33 and compileSdk 34 in your build.gradle file.

16.2 Security options

This SDK does not expose any configurable security options or parameters that could alter the behavior of the secure implementation.

The only variation allowed concerns the method of injecting the IMEAK . This operation is handled securely through one of the following standardized procedures:

1. Injecting a transport key in TR34 format (referred to as TR_IMEAK), followed by the injection of the IMEAK in TR31 format protected with that TR_IMEAK.
2. Direct injection of the IMEAK in TR34 format.

As a result, all security controls, mechanisms, and enforcement rules are statically embedded in the software design and are not subject to user selection or customization.

16.3 Application signing

To ensure the authenticity of the application, the apk must be bundled with signature.

16.4 Sensitive data deletion

The Alcineo SDK is a non isolated MPOC SDK. Indeed, the application exchanges card sensitive sensitive data with the SDK during a transaction.

Then, all variables defined in methods containing sensitive data must be cleared after use, even if this sensitive data is encrypted. The deletion is performed with filling the buffer with random number.

16.5 NFC Usage

The NFC layers is initialized by the SoftPOS library following: <https://developer.android.com/reference/android/nfc/NfcAdapter> The temporary buffers used in the NFC application layers must be cleared correctly after use. The SoftPOS libraries is in charge to clear correctly its buffers.

16.6 Files protection

If the Java developers have to store files during a long time or permanently in the mobile, they have to follow security best practices below.

The developer stores all private user data within the device's internal storage, which is sand-boxed per application.

The application doesn't need to request permission to view these files, and other applications cannot access these files.

As an added security measure, when the user uninstalls an application, the device deletes all files that the application saved within its sandbox.

You must not use the external storage. This method is forbidden for all data coming from the transaction.

Other applications and users could have access to this shared data.

For more details see <https://developer.android.com/topic/security/best-practices#safe-data>.

16.7 Application Backup Forbidden

Auto Backup for Apps automatically backs up a user's data from apps that target and run on Android 6.0 or higher. Android preserves app data by uploading it to the user's Google Drive.

<https://developer.android.com/guide/topics/data/autobackup#java>

The application developpers must configure correctly the manifest file to forbid the application backup.

You can disable backups by setting android:allowBackup to false.

This allows to avoid to backup an old state of the application with old keystore and others configuration files.

The application deletion must result in the deletion of all associated data from the COTS device.

16.8 Developer mode protection

The application must not be executed in developer mode.

This opens a well known attack path which consists of the following steps:

- activate the developer mode on the mobile
- activate the option “show touch” in the developer menu
- record the screen activities during the transaction

Moreover, this mode allows also to perform attacks through *adb* command.

To forbid the developer mode, The Alcneo’s **SDK** checks it during the runtime at a frequency of 1 seconds.

Keep inform the final user to not have enable the developer mode to use the application.

This feature anti developer mode is present only in the protected SDK build.

So, the application developer will develop its application with the unprotected SDK version to not be blocked.

Only, the protected SDK must be deployed on the field.

16.9 Screenshot Protection

If a foreground application takes a screenshot during the PINpad display , this gives to an attacker a digit position.

This is a leak of information. It could be an element to a complete attack on the PIN.

Android interfaces propose to initiate the layout with the parameter `FLAG_SECURE`.

<https://developer.android.com/reference/android/view/WindowManager.LayoutParams>

The SoftPOS library correctly sets this flag. The application does not have to manage this aspect.

16.10 Camera Protection

The library SoftPOS detects if the camera is used by a third application.

It returns an exception to the application. The native library will not be launched.

So, the application can not launch a transaction if the camera is used.

This avoids to an attacker to gain access to the cardholder CVV.

16.11 Random Generator Initialization

The random generator takes an important place in the libraries security.

For example, it is used in the attestation protocol and also to delete securely buffers countaining temporaly sensitive data.

The SDK follows the specification SP800-90A as demanded by MPOC. The DRBG type used is AES 128 CTR.

To ensure a good quality of the random generator, an entropy must be injected regularly in the SDK.

The SoftPOS SDK mange reseeded at each start and each 24h with attestation server exchange.

The confidentiality of the sensitive data included in the DRBG (key, vector) is managed by the white-box FK_DRBG.

16.12 Terminal settings

The customer shall implement a way to protect the authenticity of the kernel configuration files.

These files are sensitive, they manage which sensitive data must be encrypted and signed.

They also contain floor limits, online PIN capabilities etc..

Please note that only online PIN CVM shall be supported and then configured in terminal settings.

16.13 Attestation system

In addition to the attestation system managed by the SoftPOS library, the customer shall reenforce the security by using:

16.13.1 Attestation ID

To improve the security of the whole system, each attestation has a unique attestation ID.

The payment back-end shall check if the attestation linked to a transaction has been really successfully performed.

Please refer to *Host to Host* section for more details.

16.13.2 Periodic Attestation

In our solution one attestation request is sent by transaction.

By consequence if the final user uses a lot the SoftPOS application the security of its application will be checked regularly.

On the other hand if the application is not used so frequently it is recommended to use the deviceCheck API, see below.

16.13.3 Device Check

After suspected security problem or for customer support it could be necessary from the backend to request the application to perform a device check using deviceCheck API.

The frequency of these attestations combined with a good policy of events management will guarantee a good level of security.

This API allows to the SDK to send device information to the A&M server.

Moreover, by receiving this correctly formatted and signed request, the A&M server has good assurance of the device security.

The application can also add custom data to transmit with a JSOM file (cf. Java doc for further details).

Beware, in Alcineo solution, the A&M server must NOT receive or store sensitive authentication data, as for example PAN, PIN, but also no email, adress, phone number.

The application developer have to take care at this point to comply with to MPOC security requirements.

```
final byte[] attestationID = DeviceCheckAPI.deviceCheck(customData);
```

16.13.4 Incident API

An API is available to block a device when an attack is suspected on the application side.

```
IncidentAPI.incident();
```

On the native libraries side, this command incident is sent automatically. It is sent due to security issues. Active checks are performed by the obfuscation and code protection tool during runtime to detect tamperers. These checks are:

Check Type	Effect
Verify system properties	Detects if non-default system properties are defined
Verify system files	Detects blacklisted files in the system
Verify directory permissions	Verifies that system files have the appropriate read/write permissions
Verify packages and apks	Detects any blacklisted packages or apks on the device
Verify address space	Detects any blacklisted/hook symbols in the process' space
Verify executables	Detects SUID executables
Verify real hardware	Detects whether the app is running on an emulator
Verify runtime resources	Detects any blacklisted sockets that are used by rooting kits

For all the previous checks, if an error is detected, the A&M blocked the device DUID after receiving the incident command. Then, the device can not perform a transaction anymore.

Moreover, if the code execution is performed with error, an incident code is sent to the A&M server.

Function of its severity the A&M can blocked the device.

Some incidents have a medium severity, they block the device only if several occurrences are sent. The threshold is set by default to five.

The critical incidents are linked to the tamper detection, key management, data and PIN encipherment, session key generation and selftests. The medium incidents are linked to the key injection, enrollment, attestation and audit logs.

Below the list of the error code, their origins and severities already implemented in the libraries.

Regarding, the error codes coming from the pinpad, the error codes begin with the most significant byte set to 0x10 to be able to distinguish with those from the payment library.

It exists also error codes coming from the server itself (audit logs checks), the most significant byte is set in this case to 0x20.

Error code	Origin	Type	Severity	Floor Limit
0x0001	Protection tool tamper detection	Tamper	Critical	1
0x0002	Self-test, payment	Self-test	Critical	1
0x1002	Self-test, pinpad	Self-test	Critical	1
0x0003	Session key generation, payment	Key Management	Critical	1
0x1003	Session key generation, pinpad	Key Management	Critical	1
0x0004	Key injection, payment	Key injection	Medium	5 (*)
0x1004	Key injection, pinpad	Key injection	Medium	5 (*)
0x0005	Payment data encipherment	Cypher data	Critical	1
0x1005	PIN encipherment	Cypher data (PIN)	Critical	1
0x0006	Enrollment request	Enrollment	Medium	5
0x0016	Enrollment timeout	Enrollment	Medium	Unlimited
0x0026	Enrollment already done	Enrollment	Medium	Unlimited
0x0007	Enrollment response	Enrollment	Medium	5 (*)
0x0036	Update Initial Session Key request	Update Init Ses	Medium	5 (*)
0x0046	Update Initial Session Key timeout	Update Init Ses	Medium	Unlimited
0x0037	Update Initial Session Key response	Update Init Ses	Medium	5 (*)
0x0008	Attestation request, payment	Attestation	Medium	5 (*)
0x0018	Attestation timeout	Attestation	Medium	Unlimited
0x1008	Attestation request, pinpad	Attestation	Medium	5 (*)
0x1018	Attestation timeout, pinpad	Attestation	Medium	Unlimited

Continued on next page

Table 2 – continued from previous page

Error code	Origin	Type	Severity	Floor Limit
0x0009	Attestation response, payment	Attestation	Medium	5 (*)
0x1009	Attestation response, pinpad	Attestation	Medium	5 (*)
0x000A	Audit log writing	Audit log	Medium	5 (*)
0x000B	Audit log signature	Audit log	Medium	5 (*)
0x000C	Audit log absent	Audit log	Medium	5 (*)
0x001C	Send Audit log request	Audit log	Medium	Unlimited
0x002C	Send Audit log response	Audit log	Medium	Unlimited
0x200A	Bad global audit log signature	Audit log	Medium	3 (**)
0x200B	Bad line audit log signature	Audit log	Medium	3 (**)
0x200C	Break in audit log continuity	Audit log	Medium	3 (**)
0x000D	Master key management	Master key	Critical	1
0x0050	DRBG entropy file creation	DRBG	Medium	5 (*)
0x0051	DRBG entropy integrity problem	DRBG	Critical	1
0x0052	DRBG entropy length	DRBG	Critical	1
0x0053	DRBG context integrity problem	DRBG	Critical	1
0x0054	System security check	Self-test	Medium	Unlimited
0x000F	Incident detected by the application	A pplication	Critical	1

(*) 5 is a default value defined on server side

(**) 3 is a default value defined on server side

16.13.5 Communication with servers

The communication between application and A&M server is managed and protected by the SoftPOS library. The communication is secured with HTTPS protocol.

Moreover, under the HTTPS protocol layer, a specific protocol described in A&M Protocol Documentation is implemented for all the exchange between mobile SDKs and A&M server.

On the other side the communication between the application and the customer's backend system shall be protected as well.

The following requirements are mandatory:

- TLS version 1.2 minimum (version 1.3 is highly recommended)
- SSL Pinning

The SSL pinning allows to the application to prevent communications with the servers from being eavesdropped by an attacker performing a MITM attack.

The final secured application must not supported the unsecure HTTP protocol. For this, set in the application AndroidManifest.xml file the configuration to `android:usesCleartextTraffic="false"`.

16.13.6 Applicative protection

Data security and payment processes are ensured by the SoftPOS libraries.

However, to comply with MPOC requirements, all communications with the payment backend must pass through a secure channel.

Establishing this tunnel is the responsibility of the client application.

16.13.7 Android Version Whitelist

the A&M server embeds in its data base a whitelist of the authorized Android versions and its security patches.

The application manager is in charge of updating this whitelist through the front end access (hosted by Alcineo).

See the document Front End User Guide.

So it is demanded to the application manager to update regularly this whitelist with excluding the too old Android version and to old security patches.

For this, the application manager can consult the website: <https://source.android.com/docs/security/bulletin?hl=en>

16.13.8 Production Version Blacklist

the A&M server embeds in its data base a blacklist of the unauthorized application production versions.

The baseline OS must be reviewed for vulnerabilities at least every three months, and mitigations applied.

As a result the whilelist of the authorized Android versions and its security patches must be updated and our customers informed.

16.14 SideLoading protection

An attacker could modify the application and install a tampered version via sideloading or an alternative Play Store.

Develop protection against this attack is the responsibility of the application developer.

16.15 Repackaging protection

Even if the SDK is protected against modification and embeds tamper detection, the final application must be also protected against repackaging. APK integrity protection and digital signatures are crucial security measures that help prevent tampering, malware injection, unauthorized modifications, and fraud.

Develop protection against application repackaging is the responsibility of the application developer.

17 Key details

Key Name	Purpose/Usage	Unique per	Algorithm and size	Crypto period	Key generation method	Key erasure
FK_AK	Factory key used to derive ENC_SES and SIGN_SES for type A commands (Enrollment and Update Initial Session Key)	Per SDK production version	CMAC AES-128	4 weeks	Generated in factory using AWS KMS	Erased and replaced with each SDK application installation
FK_NONCE	Factory key used to encrypt NONCE during Enrollment and Update Initial Session Key	Per SDK production version	AES-128 ECB	4 weeks	Generated in factory using AWS KMS	Erased and replaced with each SDK application installation
TR_IPEK	Key used to inject IPEK using TR31 method D (option 1)	Injection	CMAC AES-128	4 weeks	Generated with AWS HSM	Erased after initial IPEK injection
TR_IMEAK	Key used to inject IMEAK using TR31 method D (option 1)	Injection	CMAC AES-128	4 weeks	Generated with AWS HSM	Erased after initial IMEAK injection
RSA_TR34_KRD	SDK RSA key pair (private/public) for TR34 (option 1 and 2)	Injection	RSA 3072-bit OAEP	4 weeks	Generated via OpenSSL interface	Erased at the end of TR34 injection
RSA_TR34_KDH	Host RSA public key for TR34 (option 1 and 2)	Injection	RSA 3072-bit signature	N/A	Customer dependent	Erased at the end of TR34 injection

Continued on next page

Table 3 – continued from previous page

Key Name	Purpose/Usage	Unique per	Algo-rithm and size	Crypto period	Key generation method	Key erasure
IPEK	Initial key for DUKPT session key derivation (PEKs)	Per device	AES-128 ECB	Until 32 PEKs are derived	Generated on distributor's HSM using RNG	Erased after PEK derivation
PEK	Session key used to encrypt PIN blocks	Per transaction	AES-128 ECB	Transaction duration	Derived from IPEK (or PEK)	Erased after use or max 4 weeks
IMEAK	Initial key for session keys to encrypt/sign payment data (MEAKs)	Per transaction	AES-128 ECB	Until 32 MEAKs are derived	Generated on distributor's HSM using RNG	Erased after MEAK derivation
MEAK	Session key used to derive MEK and MAK	Per transaction	AES-128 ECB	Transaction duration	Derived from IMEAK	Erased after use or max 4 weeks
MAK	Signature key for payment data	Per transaction	CMAC AES-128	Transaction duration	Derived from MEAK	Erased after use

Continued on next page

Table 3 – continued from previous page

Key Name	Purpose/Usage	Unique per	Algo-rithm and size	Crypto period	Key generation method	Key erasure
MEK	Encryption key for payment data	Per trans-action	AES-128-CBC with unpre-dictable IV	Trans-action duration	Derived from MEAK	Erased after use

18 Secure Software Configuration

To comply with D4.1.2 this integration guide includes detailed instructions about how to securely install, configure, and maintain all software components and supported platforms. Please find in the following array the list of these instructions.

Configurations	Comments
The application must use the Protected version of the SDK libraries	The unprotected versions must be used uniquely during the development phase.
The baseline is Android version 13, android api 33 or above.	
In the Application build.gradle set the following variables	• minSDK 33 (or greater) • compileSdk 34 (or greater)
The device must have hardware-backed key-store.	as TEE or Strongbox.
Permission needed in application manifest file	• ACCESS_NETWORK_STATE • INTERNET • READ_EXTERNAL_STORAGE
Dependencies needed :	This dependencies are needed in the application to work correctly with the SDK.

Continued on next page

Table 4 – continued from previous page

Configurations	Comments
<code>'com.google.guava:guava'</code> <code>'com.neovisionaries:nv-i18n'</code> <code>'org.jetbrains:annotations'</code> implementation <code>'androidx.lifecycle:lifecycle-process'</code> <code>'com.squareup.retrofit2:retrofit'</code> Group : <code>'com.squareup.retrofit2'</code> , name: <code>'converter-jackson'</code>	
The application must call the SoftPos APIs as described in Integration guide	• <code>AttestationAgent.init</code> • <code>SecurityAPI.initialize</code> • <code>PinpadAPI.initialize</code> • <code>UpdateInitialSessionKeyAPI.updateInitSessionKey</code> • <code>EnrollmentAPI.newEnrollment</code> • <code>RkiExample.tr34IMEAKInjection</code> For more details see the dedicated paragraphs in Integration Guide.
The Payment back end system must be able to generate a TR34 block for key injection of Initial DUKPT keys	The version followed by the SDK in the TR34 2012
Two injection keys must be performed:	• IMEAK injection for transaction data • IPEK for PIN enciphered
IMEAK must be an AES 128 bits key	
IPEK must be an AES 128 bits key	
The application must check the terminal setting authenticity.	Terminal setting are included in <code>/assets/settings/</code> of the application data.

Continued on next page

Table 4 – continued from previous page

Configurations	Comments
The tag 0xDFDF33 in terminal setting must be filled with sensitive data tags.	
The SDK new build generated every 4 weeks must be included in the Application every 4 weeks.	This guaranties that the whiteboxes keys are different.
The application must be signed.	This avoids that an attacker changes the application to obtain an unexpected behavior.
If the application stored data, this must be done in the application sandbox. Other application must not have access to the payment application data.	See for more details https://developer.android.com/privacy-and-security/security-tips?hl=fr#safe-data
Set in application manisfest file	The application backup must be forbidden.
<code>android:allowBackup="false"</code>	
The application must be installed on a device which does not allow the developer mode.	
If the application detects a problem incompatible to its secure execution it must call the interface.	Even if the SDK make its owns security checks, the application can detect other security problems linked to its activity and responsibilities <code>IncidentAPI.incident()</code> ;
The application and customer backend system must be connected through a TLS connection:	• TLS version 1.2 minimum (version 1.3 is highly recommended) • SSL Pinning
The customer is in charge to update the whitelist in the A&M server database through its frontend access.	It excludes the too-old security patches and accepts the newest.

Continued on next page

Table 4 – continued from previous page

Configurations	Comments
The application developer is in charge to Side-Loading protection.	
The final application must be protected against repackaging.	The application signature is an important part of the protection, but also obfuscation of the application bytecode and inclusion of tamper detection with a specialized tool.
The customer payment backend system have to check the attestation ID to link each transaction to a transaction.	See the chapter Host to Host of the security guideline.

19 Host to Host

19.1 Attestation ID API

To improve the security of the whole system, each attestation has a unique attestation ID. The payment back-end shall check if the attestation linked to a transaction has been really successfully performed.

To implement such API for the payment back-end you need to use the table `payment_requests`. This table stores the attestation identifier in `attestation_id`. Request could be something like :

```
select      count(*)      from      payment_requests      where      attestation_id      =  
'98C9AB3B0801CF63B2EEF1CBCD858E79F6B3FD5081CAF47122038E230C9F2FBB'
```